The 8085 Master Manual

Volume IV: Expanding the Universe

(Address Decoding & Interfacing)

A Grassroots Engineering Guide

Constructing the bridge between the Microprocessor and the Physical World.

"An interface is a boundary across which two independent entities meet and communicate."

Chapter 1: The Philosophy of Interfacing

1.1 The Isolation of the Core

Up to this point, we have meticulously constructed the internal architecture of the 8085 microprocessor. We have established its registers, its arithmetic logic unit, and its rigorous timing sequencers. However, a microprocessor in isolation is entirely useless. It is a brain in a jar. It possesses immense processing capability but has no memory to store programs permanently and no means to interact with human operators or physical sensors.

To build a functional computer, we must connect the 8085 to external components:

- **Memory Devices:** Read-Only Memory (ROM) to store permanent boot instructions, and Random Access Memory (RAM) to store temporary variables and user programs.
- **Peripheral Devices (I/O):** Keyboards, displays, analog-to-digital converters (ADC), and sensors.

1.2 Defining the Interface

The act of connecting these external entities to the microprocessor is called **Interfacing**.

> **THE DEFINITION OF INTERFACING**
>
> Interfacing is the concept and physical implementation of an interaction point between components. In the 8085, it is the hardware circuitry (and associated software protocols) that matches the stringent timing, electrical, and logical signal requirements of the external memory and I/O devices with the signals provided by the microprocessor's System Bus.

The primary function of the microprocessor is to accept data from input devices, read data from memory, process that data, and send the results to output devices. Because memory chips and peripheral controllers operate at different speeds and possess different internal architectures than the CPU, we cannot simply solder their pins directly to the CPU's pins. We must build a logical bridge.

Chapter 2: The Anatomy of a Memory Chip

Before we can interface memory to the microprocessor, we must intimately understand what a memory chip expects from the outside world.

2.1 Signal Requirements of Memory

Whether a chip is RAM or EPROM, it physically consists of an immense grid of microscopic storage registers. To successfully read from or write to a specific register, the memory chip demands three distinct types of electrical information, perfectly mirroring the CPU's System Bus:

1. **Address Lines** (A_0 - A_n)**:** To select a specific internal register.
2. **Data Lines** (D_0 - D_7)**:** The physical pathway for the 8-bit data to enter or leave the chip.
3. **Control Lines:** Signals that enable the chip and dictate the direction of data flow.

2.2 The Mathematics of Capacity

The storage capacity of a memory chip dictates exactly how many Address Lines it must possess. The relationship is governed by the formula: C = 2^N, where C is the number of internal registers and N is the number of address pins.

Example 1: A 2048-Byte (2 KB) RAM Chip
To uniquely identify 2048 distinct registers, the chip requires \log_2(2048) = 11 address lines. These are labelled A_0 through A_{10}.

Example 2: A 4096-Byte (4 KB) EPROM Chip
To identify 4096 registers, the chip requires \log_2(4096) = 12 address lines. These are labelled A_0 through A_{11}.

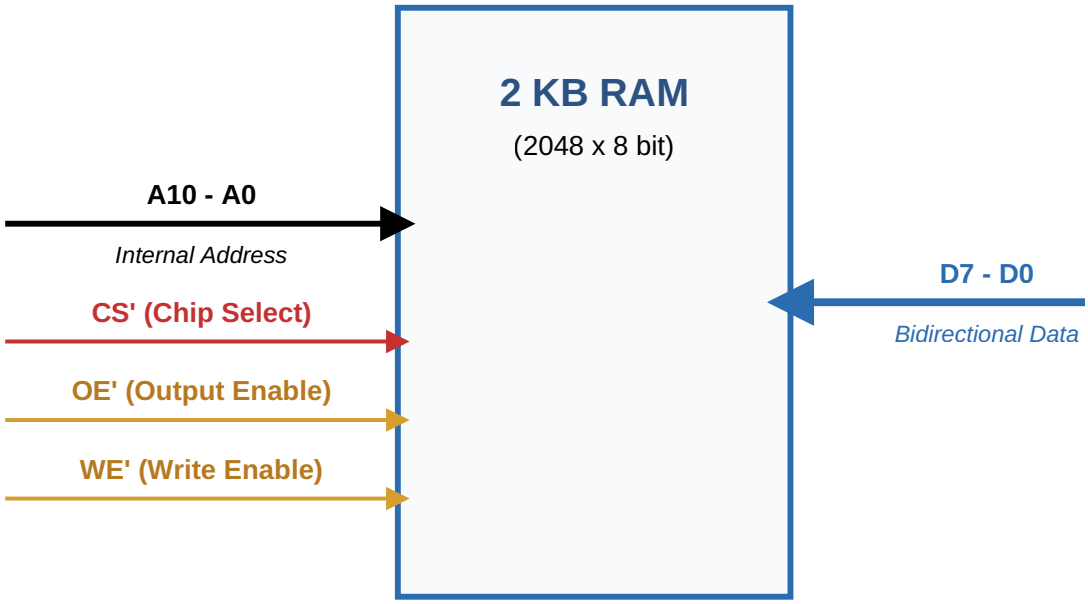
2.3 The Control Pins (CS', OE', WE')

A memory chip does not permanently broadcast data onto the data bus. If it did, it would instantly cause an electrical short-circuit the moment another chip attempted to broadcast. Memory chips use internal *tri-state buffers* to disconnect themselves from the data bus until explicitly commanded to connect.

- **Chip Select (CS') or Chip Enable (CE'):** This is the master switch of the memory chip. It is an active-low pin. If CS' = 1, the entire chip is asleep, and its data pins are in a high-impedance (disconnected) state. If CS' = 0, the chip wakes up and pays attention to the address and control lines.

- **Output Enable (OE'):** Active-low. When CS' = 0 and OE' = 0, the chip pushes the data from the selected register out onto the Data Bus (A Read Operation).
- **Write Enable (WE'):** Active-low. Found only on RAM chips. When CS' = 0 and WE' = 0, the chip absorbs the data currently sitting on the Data Bus and stores it into the selected register (A Write Operation).

2.4 Visualizing the Generic Memory Interfacing Block



The CPU outputs a 16-bit address (A_0 - A_{15}). However, our 2 KB RAM chip only has 11 address pins (A_0 - A_{10}). We wire the CPU's lower 11 address lines directly to the memory chip's 11 address pins.

The Central Question of Interfacing: What do we do with the remaining 5 address lines from the CPU (A_{11}, A_{12}, A_{13}, A_{14}, A_{15})?

We cannot ignore them. If we ignore them, the RAM chip will respond any time the lower 11 bits match, regardless of what the upper 5 bits are. To place this RAM chip at one specific, unique 64KB location, we must use the remaining 5

lines to control the RAM's **Chip Select** (CS') pin. This process is called **Address Decoding**.

Chapter 3: The Imperative of Address Decoding

The Bus Contention Crisis

Suppose a motherboard has four 2 KB RAM chips wired directly to the Data Bus and the lower 11 Address Lines. If the CPU requests data from address `0005H`, all four chips will see `0005H` on their lower pins. All four chips will simultaneously wake up, fetch the data in their 5th register, and attempt to shove it onto the Data Bus at the exact same microsecond.

This creates an electrical short-circuit known as **Bus Contention**. The CPU receives garbage data, and the hardware may be physically damaged.

To prevent Bus Contention, the system must guarantee that **only one memory chip's** CS' **pin is driven LOW at any given time**.

Address decoding is the combinational logic circuitry placed between the CPU's high-order address lines and the memory chips' CS' pins. It acts as a strict geographical zoning authority.

3.1 Decoding via NAND Logic

The simplest form of address decoding utilizes fundamental logic gates, predominantly the NAND gate. Because the CS' pin requires a LOW (0) signal to activate the chip, and a NAND gate naturally outputs a LOW only when all its inputs are HIGH (1), it is the perfect tool for the job.

EXAMPLE: DECODING A 2 KB RAM TO BASE ADDRESS 2000H

We have a 2 KB RAM (Requires A_0 - A_{10}).

Remaining lines for decoding: A_{15}, A_{14}, A_{13}, A_{12}, A_{11}.

We want the chip to activate when the address is in the `2000H` range.

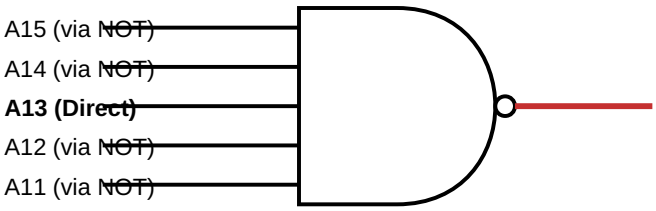
Let us examine `2000H` in binary:

A15	A14	A13	A12	A11	A10 - A0
0	0	1	0	0	(Internal chip address)

For the CS' pin to receive a 0, the NAND gate must output a 0. A NAND gate outputs 0 only when all inputs are 1. Therefore, we must invert the zeros before feeding them into the NAND gate.

LOGIC EQUATION:

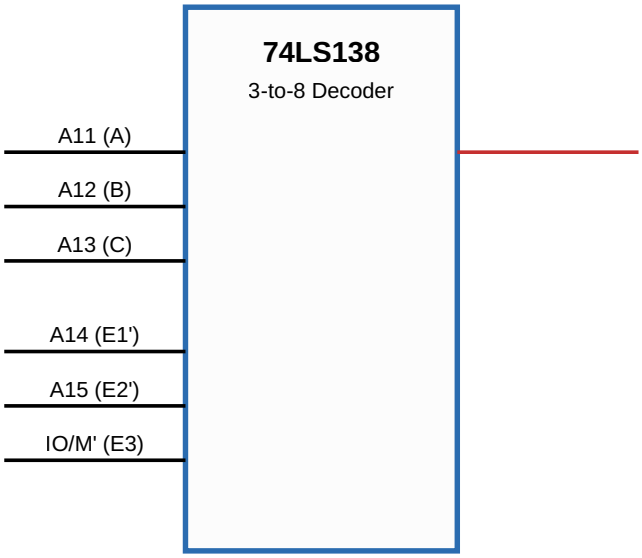
CS' = \overline{\overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot \overline{A_{12}} \cdot \overline{A_{11}}}


A15 (via NOT)
A14 (via NOT)
A13 (Direct)
A12 (via NOT)
A11 (via NOT)

3.2 Decoding via the 74LS138 (3-to-8 Decoder)

Using discrete NAND and NOT gates becomes excessively bulky and expensive when a system requires multiple memory chips. Engineers circumvent this by utilizing highly integrated decoding ICs, the standard being the **74LS138 3-to-8 line decoder**.

The 74LS138 accepts 3 binary input lines (A, B, C) and activates exactly one of its 8 output lines (Y_0' to Y_7'). Crucially, the outputs are active-low, perfectly matching the CS' requirements of memory chips. It also features three Enable pins (E_1', E_2', E_3) which can be tied to the highest-order address lines or system control signals (like IO/M') to activate the decoder only under specific conditions.



This elegant configuration allows a single 74LS138 chip to independently select up to eight different 2 KB RAM chips, mapping out 16 KB of contiguous memory space without any risk of bus contention. The IO/M' connection to E_3 ensures these RAM chips are absolutely blind to any I/O commands sent by the CPU.

Chapter 4: Absolute vs. Partial Decoding

The mathematical rigor with which an engineer applies address decoding directly affects the physical layout of the memory map. The MH Methodology rigorously distinguishes between two paradigms: Absolute and Partial Decoding.

4.1 Absolute Decoding (Exhaustive Decoding)

In **Absolute Decoding**, every single one of the 16 address lines is explicitly accounted for.

- The lower N lines connect directly to the memory chip to select internal registers.
- **All** remaining (16 - N) lines are routed into the decoding logic to generate the CS' signal.

Because no address lines are left un-decoded, the memory chip responds to one, and *only one*, specific range of addresses in the 64 KB space. There is no ambiguity. This is the professional standard for robust system design.

4.2 Partial Decoding and The Phenomenon of Image Addresses

In smaller, cost-sensitive embedded systems, engineers may choose to leave one or more of the higher-order address lines completely unconnected. This is **Partial Decoding**.

When an address line is unconnected to the decoder, the decoder treats it as a "Don't Care" (X). The logic gates will generate the CS' signal regardless of whether the CPU drives that specific line HIGH or LOW.

The Consequence: Foldback (Image Addresses)
Because the "Don't Care" bit can be a 0 or a 1, there are now multiple distinct 16-bit addresses that will successfully trigger the exact same physical memory chip. The chip "folds back" upon itself, appearing at multiple locations within the 64 KB memory map. These duplicate, overlapping addresses are known as **Image Addresses**.

Mathematical Derivation of Image Addresses

Let us perform a strict derivation according to the MH methodology.

SCENARIO:

We interface a 256-Byte RAM chip.
A 256-Byte chip requires 8 address lines (A_0 - A_7) to select its internal registers.
We design a partial decoder that only checks three lines: A_{15}=0, A_{14}=0, A_{13}=1.
The remaining lines (A_{12}, A_{11}, A_{10}, A_9, A_8) are left unconnected (Don't Cares).

Let us map the address bit format:

A15	A14	A13	A12	A11	A10	A9	A8	A7...A0
0	0	1	X	X	X	X	X	Internal

CALCULATING THE BASE ADDRESS:

The Base Address is defined as the range when all "Don't Care" bits are forced to 0.

Start: 0010 0000 0000 0000 = 2000H
End: 0010 0000 1111 1111 = 20FFH

CALCULATING IMAGE ADDRESS 1:

What if the CPU accidentally flips A_8 to 1? The decoder only looks at A15-A13, so it still activates the chip.

Start: 0010 0001 0000 0000 = 2100H

End: 0010 0001 1111 1111 = 21FFH

Writing to `2100H` will physically overwrite the data sitting at `2000H`, because they are the exact same physical silicon register.

CALCULATING TOTAL IMAGE ADDRESSES:

There are 5 unconnected address lines.

Total number of duplicate mappings = 2^5 = 32.

This single 256-Byte chip will occupy 32 different spaces (consuming 8 KB of total address space) inside the 8085's memory map. This illustrates why partial decoding is dangerous in large systems.

Chapter 5: I/O Interfacing - Bridging the External World

The microprocessor must communicate not only with memory but with the physical world—reading keyboards and illuminating displays. These devices are interfaced through Input/Output (I/O) ports.

The 8085 architecture supports two distinct paradigms for I/O interfacing. Understanding the dichotomy between them is a fundamental requirement of the syllabus.

5.1 Peripheral Mapped I/O (I/O Mapped I/O)

In this scheme, the Intel engineers granted I/O devices their own, exclusive address space, entirely separate from the 64 KB memory map.

- **Address Space:** I/O ports are assigned an **8-bit address** (ranging from `00H` to `FFH`). This allows for a maximum of 256 Input ports and 256 Output ports.
- **Bus Behavior:** When an I/O instruction is executed, the 8085 takes the 8-bit port address and duplicates it on both the lower bus (AD_0-AD_7) and the upper bus (A_8-A_{15}).

- **Control Signals:** The CPU drives the IO/M' **pin HIGH (1)**. The decoding logic uses this to generate the specific **IOR'** and **IOW'** signals.
- **Software Instructions:** The programmer must use the dedicated, 2-byte instructions: `IN <8-bit Port Address>` and `OUT <8-bit Port Address>`. These instructions inherently transfer data exclusively between the targeted port and the Accumulator.

5.2 Memory Mapped I/O

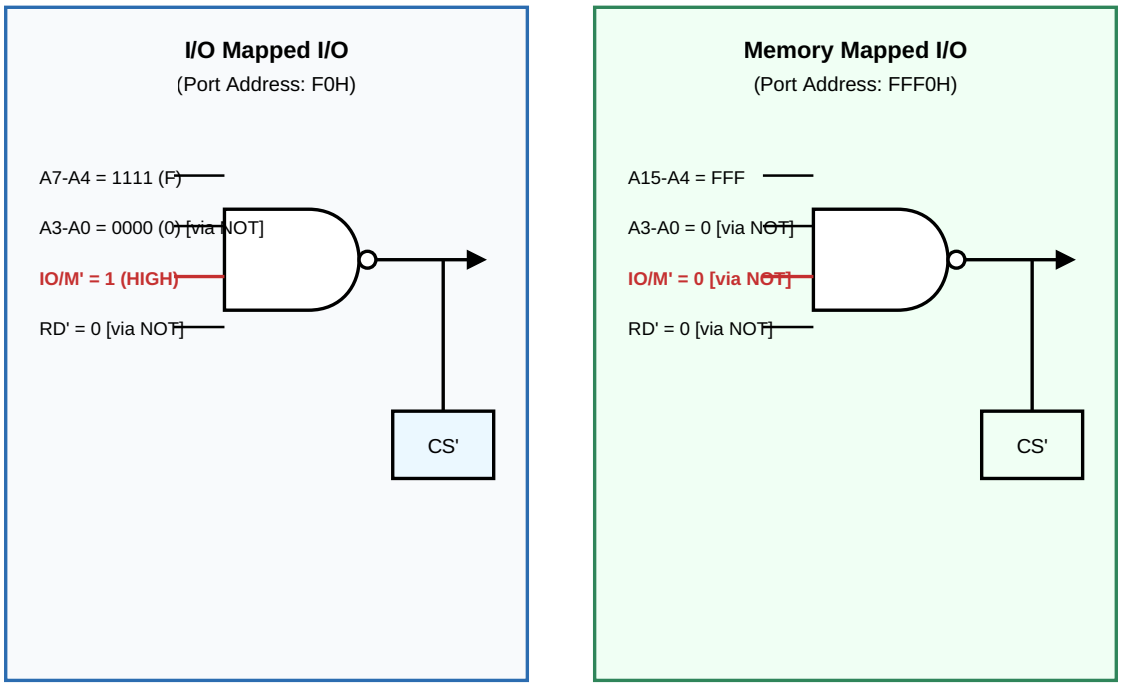
In Memory Mapped I/O, the external peripheral is structurally "disguised" as a memory chip. The hardware decoder wires the I/O port directly into the standard 64 KB memory space.

- **Address Space:** The I/O port is assigned a full **16-bit address** (e.g., `FFF0H`). It consumes a slot in the memory map; a RAM chip cannot be placed at this address.
- **Control Signals:** Because the CPU believes it is talking to memory, it drives the IO/M' **pin LOW (0)**. The decoding logic utilizes the standard **MEMR'** and **MEMW'** signals.
- **Software Instructions:** The programmer can use *any* instruction that references memory to interact with the I/O port (e.g., `LDA FFF0H`, `STA FFF0H`, `MOV A, M`, `ADD M`). This provides immense programming flexibility, allowing data to be pulled directly from a port into a register other than the Accumulator, or even mathematically manipulated directly at the port.

5.3 Visualizing the Interfacing Dichotomy

The physical circuitry required to decode these two different architectures highlights their fundamental differences.

Comparative Decoding Logic (Addressing an I/O Port at F0H vs FFF0H)



Observe the strict structural differences. In I/O Mapped I/O, the decoder logic only monitors the lower 8 address lines and looks for the `IO/M'` pin to be HIGH. In Memory Mapped I/O, the decoder must monitor all 16 address lines to isolate the device within the 64KB grid, and it looks for the `IO/M'` pin to be LOW.

This concludes Volume IV. We have successfully constructed the hardware bridge that connects the internal logic of the 8085 to the physical memory and peripheral devices on the motherboard. The hardware architecture is now complete. The final step in mastering the 8085 is learning to command it through software—the art of Assembly Language Programming.

End of Volume IV